SOLID Principles

Complete guide to SOLID principles with practical examples and real-world applications.

What is SOLID?

SOLID = Five design principles for writing maintainable, scalable object-oriented code

Created by: Robert C. Martin (Uncle Bob)

Purpose:

- Make code easier to understand
- Make code easier to maintain
- Make code easier to extend
- Reduce bugs from changes

The Five Principles:

1. **S**ingle Responsibility Principle
2. **O**pen/Closed Principle
3. **L**iskov Substitution Principle
4. **I**nterface Segregation Principle
5. **D**ependency Inversion Principle

---

1. Single Responsibility Principle (SRP)

Definition: A class should have only ONE reason to change

In other words: A class should do ONE thing and do it well

Bad Example (Violates SRP)

```
class User:
    def __init__(self, name, email):
        self.name = name
        self.email = email

    def get_user_data(self):
        """Business logic"""
        return {"name": self.name, "email": self.email}

    def save_to_database(self):
        """Database logic"""
        db.execute("INSERT INTO users VALUES (?, ?)", (self.name, self.email))

    def send_welcome_email(self):
        """Email logic"""
```

```
        smtp.send(self.email, "Welcome!", f"Hi {self.name}")

    def generate_report(self):
        """Reporting logic"""
        return f"User Report: {self.name} - {self.email}"
```

Problems:

- User class has 4 responsibilities (data, database, email, reporting)
- Change in email format requires changing User class
- Change in database schema requires changing User class
- Hard to test (need database and email server)

Good Example (Follows SRP)

```
# 1. User class - only handles user data
class User:
    def __init__(self, name, email):
        self.name = name
        self.email = email

    def get_data(self):
        return {"name": self.name, "email": self.email}

# 2. UserRepository - handles database
class UserRepository:
    def save(self, user: User):
        db.execute(
            "INSERT INTO users VALUES (?, ?)",
            (user.name, user.email)
        )

    def find_by_email(self, email: str):
        result = db.query("SELECT * FROM users WHERE email = ?", (email,))
        return User(result['name'], result['email'])

# 3. EmailService - handles emails
class EmailService:
    def send_welcome_email(self, user: User):
        smtp.send(
            user.email,
            "Welcome!",
            f"Hi {user.name}"
        )

# 4. UserReportGenerator - handles reports
class UserReportGenerator:
    def generate(self, user: User):
        return f"User Report: {user.name} - {user.email}"

# Usage
```

```
user = User("Alice", "alice@example.com")
UserRepository().save(user)
EmailService().send_welcome_email(user)
report = UserReportGenerator().generate(user)
```

Benefits: ☑ Each class has one responsibility ☑ Easy to test (can mock dependencies) ☑ Changes are isolated ☑ Can reuse EmailService for other entities

Real-World Example: E-commerce Order

```python
# BAD: One class does everything
class Order:
    def calculate_total(self): pass
    def validate_items(self): pass
    def save_to_database(self): pass
    def send_confirmation_email(self): pass
    def process_payment(self): pass
    def update_inventory(self): pass
    def generate_invoice_pdf(self): pass

# GOOD: Separate responsibilities
class Order:
    """Only holds order data"""
    def __init__(self, items, user_id):
        self.items = items
        self.user_id = user_id

class OrderCalculator:
    """Calculates totals, taxes, discounts"""
    def calculate_total(self, order: Order): pass

class OrderValidator:
    """Validates order data"""
    def validate(self, order: Order): pass

class OrderRepository:
    """Database operations"""
    def save(self, order: Order): pass

class OrderEmailService:
    """Email notifications"""
    def send_confirmation(self, order: Order): pass

class PaymentProcessor:
    """Payment processing"""
    def process(self, order: Order): pass

class InventoryUpdater:
    """Inventory management"""
    def update_stock(self, order: Order): pass
```

```
class InvoiceGenerator:
    """PDF generation"""
    def generate(self, order: Order): pass
```

---

2. Open/Closed Principle (OCP)

Definition: Classes should be OPEN for extension but CLOSED for modification

In other words: Add new functionality by extending, not by modifying existing code

Bad Example (Violates OCP)

```
class PaymentProcessor:
    def process_payment(self, payment_type, amount):
        if payment_type == "credit_card":
            print(f"Processing credit card payment: ${amount}")
            # Credit card logic
        elif payment_type == "paypal":
            print(f"Processing PayPal payment: ${amount}")
            # PayPal logic
        elif payment_type == "crypto":
            print(f"Processing crypto payment: ${amount}")
            # Crypto logic
        # Add more payment types? Must modify this class!
```

Problems:

- Adding new payment type requires modifying existing code
- Risk of breaking existing functionality
- Violates OCP (class not closed for modification)

Good Example (Follows OCP)

```
from abc import ABC, abstractmethod

# Abstract base - defines contract
class PaymentMethod(ABC):
    @abstractmethod
    def process(self, amount: float):
        pass

# Concrete implementations
class CreditCardPayment(PaymentMethod):
    def process(self, amount: float):
        print(f"Processing credit card: ${amount}")
        # Credit card logic

class PayPalPayment(PaymentMethod):
```

```
    def process(self, amount: float):
        print(f"Processing PayPal: ${amount}")
        # PayPal logic

class CryptoPayment(PaymentMethod):
    def process(self, amount: float):
        print(f"Processing crypto: ${amount}")
        # Crypto logic

# Processor doesn't need modification for new payment types
class PaymentProcessor:
    def process_payment(self, payment_method: PaymentMethod, amount: float):
        payment_method.process(amount)

# Usage - Adding new payment type doesn't modify existing code
class ApplePayPayment(PaymentMethod):  # New!
    def process(self, amount: float):
        print(f"Processing Apple Pay: ${amount}")

# Works without modifying PaymentProcessor
processor = PaymentProcessor()
processor.process_payment(CreditCardPayment(), 100)
processor.process_payment(ApplePayPayment(), 50)  # New type, no modification!
```

Benefits: ☑ Add new payment types without modifying existing code ☑ No risk of breaking existing payments ☑ Easy to test each payment type independently ☑ Follows OCP (open for extension, closed for modification)

Real-World Example: Notification System

```
# Abstract notification
class Notification(ABC):
    @abstractmethod
    def send(self, user, message):
        pass

# Existing implementations
class EmailNotification(Notification):
    def send(self, user, message):
        smtp.send(user.email, message)

class SMSNotification(Notification):
    def send(self, user, message):
        twilio.send(user.phone, message)

# Notification sender - CLOSED for modification
class NotificationSender:
    def __init__(self):
        self.notifications = []

    def add_notification_method(self, notification: Notification):
```

```
        self.notifications.append(notification)

    def notify_all(self, user, message):
        for notification in self.notifications:
            notification.send(user, message)

# EXTEND with new notification types without modifying existing code
class SlackNotification(Notification):  # New!
    def send(self, user, message):
        slack.post(user.slack_id, message)

class PushNotification(Notification):  # New!
    def send(self, user, message):
        fcm.send(user.device_token, message)

# Usage
sender = NotificationSender()
sender.add_notification_method(EmailNotification())
sender.add_notification_method(SMSNotification())
sender.add_notification_method(SlackNotification())  # Just add, don't modify!
sender.notify_all(user, "Your order shipped!")
```

---

3. Liskov Substitution Principle (LSP)

Definition: Objects of a subclass should be replaceable with objects of the superclass without breaking the application

In other words: Derived classes must be substitutable for their base classes

Bad Example (Violates LSP)

```
class Bird:
    def fly(self):
        print("Flying in the sky")

class Sparrow(Bird):
    def fly(self):
        print("Sparrow flying")

class Penguin(Bird):
    def fly(self):
        raise Exception("Penguins can't fly!")  # Breaks LSP!

# This code breaks when using Penguin
def make_bird_fly(bird: Bird):
    bird.fly()  # Expects all Birds can fly

sparrow = Sparrow()
make_bird_fly(sparrow)  # Works
```

```
penguin = Penguin()
make_bird_fly(penguin)  # Crashes! Violates LSP
```

Problem: Penguin is a Bird, but can't fly. Substituting Penguin for Bird breaks the code.

Good Example (Follows LSP)

```
class Bird:
    def eat(self):
        print("Eating")

class FlyingBird(Bird):
    def fly(self):
        print("Flying")

class FlightlessBird(Bird):
    def walk(self):
        print("Walking")

# Proper inheritance
class Sparrow(FlyingBird):
    def fly(self):
        print("Sparrow flying")

class Penguin(FlightlessBird):
    def walk(self):
        print("Penguin waddling")

# Functions work with appropriate types
def make_flying_bird_fly(bird: FlyingBird):
    bird.fly()

def make_bird_eat(bird: Bird):
    bird.eat()

# Usage
sparrow = Sparrow()
penguin = Penguin()

make_bird_eat(sparrow)   # Works
make_bird_eat(penguin)   # Works
make_flying_bird_fly(sparrow)  # Works
# make_flying_bird_fly(penguin)  # Type error - correct!
```

Benefits: ☑ Penguins and Sparrows can both be Birds ☑ Type system prevents misuse ☑ No unexpected exceptions ☑ Follows LSP

Real-World Example: Storage

```python
# BAD: Violates LSP
class Storage:
    def save(self, data):
        pass

class DatabaseStorage(Storage):
    def save(self, data):
        db.insert(data)

class ReadOnlyStorage(Storage):
    def save(self, data):
        raise Exception("Read-only!")  # Violates LSP!

# GOOD: Follows LSP
class ReadableStorage:
    def read(self, id):
        pass

class WritableStorage(ReadableStorage):
    def read(self, id):
        pass

    def save(self, data):
        pass

class DatabaseStorage(WritableStorage):
    def read(self, id):
        return db.query(id)

    def save(self, data):
        db.insert(data)

class ReadOnlyCache(ReadableStorage):
    def read(self, id):
        return cache.get(id)
    # No save() method - correct!

# Usage
def fetch_data(storage: ReadableStorage, id):
    return storage.read(id)  # Works with all ReadableStorage

def store_data(storage: WritableStorage, data):
    storage.save(data)  # Works with all WritableStorage

# Both work correctly
db = DatabaseStorage()
cache = ReadOnlyCache()

fetch_data(db, "123")      # Works
fetch_data(cache, "123")   # Works
store_data(db, {"data"})   # Works
# store_data(cache, {"data"})  # Type error - correct!
```

Rectangle-Square Problem

```python
# FAMOUS LSP VIOLATION
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def set_width(self, width):
        self.width = width

    def set_height(self, height):
        self.height = height

    def area(self):
        return self.width * self.height

class Square(Rectangle):
    def set_width(self, width):
        self.width = width
        self.height = width  # Keep square property

    def set_height(self, height):
        self.width = height
        self.height = height  # Keep square property

# This breaks!
def resize_rectangle(rect: Rectangle):
    rect.set_width(5)
    rect.set_height(4)
    assert rect.area() == 20  # Expects 5 * 4 = 20

rect = Rectangle(2, 3)
resize_rectangle(rect)  # Works, area = 20

square = Square(2, 2)
resize_rectangle(square)  # Fails! area = 16, not 20

# SOLUTION: Don't inherit Square from Rectangle
class Shape:
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

class Square(Shape):
```

```
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side * self.side
```

---

4. Interface Segregation Principle (ISP)

Definition: No client should be forced to depend on methods it doesn't use

In other words: Many small, specific interfaces are better than one large, general interface

Bad Example (Violates ISP)

```
class Worker:
    """Fat interface - forces all workers to implement everything"""
    def work(self):
        pass

    def eat(self):
        pass

    def sleep(self):
        pass

class Human(Worker):
    def work(self):
        print("Human working")

    def eat(self):
        print("Human eating")

    def sleep(self):
        print("Human sleeping")

class Robot(Worker):
    def work(self):
        print("Robot working")

    def eat(self):
        pass  # Robots don't eat! Forced to implement

    def sleep(self):
        pass  # Robots don't sleep! Forced to implement
```

Problem: Robot is forced to implement eat() and sleep() even though it doesn't need them.

Good Example (Follows ISP)

```python
# Split into smaller, focused interfaces
class Workable:
    def work(self):
        pass

class Eatable:
    def eat(self):
        pass

class Sleepable:
    def sleep(self):
        pass

# Humans implement all three
class Human(Workable, Eatable, Sleepable):
    def work(self):
        print("Human working")

    def eat(self):
        print("Human eating")

    def sleep(self):
        print("Human sleeping")

# Robots only implement what they need
class Robot(Workable):
    def work(self):
        print("Robot working")
    # No eat() or sleep() - not forced!

# Usage
def make_work(worker: Workable):
    worker.work()

def feed(eater: Eatable):
    eater.eat()

human = Human()
robot = Robot()

make_work(human)  # Works
make_work(robot)  # Works
feed(human)       # Works
# feed(robot)     # Type error - correct!
```

Benefits: ☑ Robot only implements what it needs ☑ No empty/dummy implementations ☑ Clear contracts ☑ Follows ISP

Real-World Example: Printer

```python
# BAD: Fat interface
class Printer:
    def print(self, document):
        pass

    def scan(self, document):
        pass

    def fax(self, document):
        pass

class SimplePrinter(Printer):
    def print(self, document):
        print(f"Printing {document}")

    def scan(self, document):
        raise Exception("Not supported")  # Forced to implement!

    def fax(self, document):
        raise Exception("Not supported")  # Forced to implement!

# GOOD: Segregated interfaces
class Printable:
    def print(self, document):
        pass

class Scannable:
    def scan(self, document):
        pass

class Faxable:
    def fax(self, document):
        pass

class SimplePrinter(Printable):
    def print(self, document):
        print(f"Printing {document}")

class MultiFunctionPrinter(Printable, Scannable, Faxable):
    def print(self, document):
        print(f"Printing {document}")

    def scan(self, document):
        print(f"Scanning {document}")

    def fax(self, document):
        print(f"Faxing {document}")

# Usage
def print_document(printer: Printable, doc):
    printer.print(doc)

def scan_document(scanner: Scannable, doc):
```

```
    scanner.scan(doc)

simple = SimplePrinter()
multifunction = MultiFunctionPrinter()

print_document(simple, "doc.pdf")      # Works
print_document(multifunction, "doc.pdf")  # Works
scan_document(multifunction, "image.jpg")  # Works
# scan_document(simple, "image.jpg")   # Type error - correct!
```

API Example

```
# BAD: Fat API client
class APIClient:
    def get_users(self): pass
    def create_user(self): pass
    def delete_user(self): pass
    def get_orders(self): pass
    def create_order(self): pass
    def get_products(self): pass
    def create_product(self): pass
    # ... 50 more methods

class ReadOnlyClient(APIClient):
    def get_users(self): pass
    def create_user(self): raise Exception("Read-only!")  # Forced!
    def delete_user(self): raise Exception("Read-only!")  # Forced!
    # ... all write methods throw errors

# GOOD: Segregated interfaces
class UserReader:
    def get_users(self): pass

class UserWriter:
    def create_user(self): pass
    def delete_user(self): pass

class OrderReader:
    def get_orders(self): pass

class OrderWriter:
    def create_order(self): pass

# Clients implement only what they need
class ReadOnlyClient(UserReader, OrderReader):
    def get_users(self): pass
    def get_orders(self): pass

class AdminClient(UserReader, UserWriter, OrderReader, OrderWriter):
    def get_users(self): pass
    def create_user(self): pass
```

```
    def delete_user(self): pass
    def get_orders(self): pass
    def create_order(self): pass
```

---

5. Dependency Inversion Principle (DIP)

Definition:

- High-level modules should not depend on low-level modules. Both should depend on abstractions.
- Abstractions should not depend on details. Details should depend on abstractions.

In other words: Depend on interfaces/abstractions, not concrete implementations

Bad Example (Violates DIP)

```
# Low-level module (concrete implementation)
class MySQLDatabase:
    def connect(self):
        print("Connecting to MySQL")

    def query(self, sql):
        print(f"Executing: {sql}")
        return ["result1", "result2"]

# High-level module depends on low-level module
class UserService:
    def __init__(self):
        self.db = MySQLDatabase()  # Direct dependency on concrete class

    def get_users(self):
        self.db.connect()
        return self.db.query("SELECT * FROM users")
```

Problems:

- UserService is tightly coupled to MySQLDatabase
- Can't switch to PostgreSQL without modifying UserService
- Hard to test (need real MySQL database)
- Violates DIP (high-level depends on low-level)

Good Example (Follows DIP)

```
from abc import ABC, abstractmethod

# Abstraction (interface)
class Database(ABC):
    @abstractmethod
    def connect(self):
```

```
        pass

    @abstractmethod
    def query(self, sql):
        pass

# Low-level modules implement abstraction
class MySQLDatabase(Database):
    def connect(self):
        print("Connecting to MySQL")

    def query(self, sql):
        print(f"MySQL: {sql}")
        return ["result1", "result2"]

class PostgreSQLDatabase(Database):
    def connect(self):
        print("Connecting to PostgreSQL")

    def query(self, sql):
        print(f"PostgreSQL: {sql}")
        return ["result1", "result2"]

class MockDatabase(Database):
    def connect(self):
        pass

    def query(self, sql):
        return ["mock_result"]

# High-level module depends on abstraction
class UserService:
    def __init__(self, database: Database):  # Depends on interface
        self.db = database

    def get_users(self):
        self.db.connect()
        return self.db.query("SELECT * FROM users")

# Usage - Easy to switch implementations
service1 = UserService(MySQLDatabase())
service2 = UserService(PostgreSQLDatabase())
service3 = UserService(MockDatabase())  # Testing!

users = service1.get_users()  # MySQL
users = service2.get_users()  # PostgreSQL
users = service3.get_users()  # Mock for testing
```

Benefits: ☑ UserService doesn't depend on specific database ☑ Easy to switch databases ☑ Easy to test with mock ☑ Follows DIP (both depend on abstraction)

Real-World Example: Email Service

```python
# BAD: Tight coupling
class UserRegistration:
    def __init__(self):
        self.smtp_client = SMTPClient()  # Concrete dependency

    def register_user(self, user):
        # ... register logic ...
        self.smtp_client.send(user.email, "Welcome!")

# GOOD: Dependency inversion
class EmailSender(ABC):
    @abstractmethod
    def send(self, to, message):
        pass

# Concrete implementations
class SMTPEmailSender(EmailSender):
    def send(self, to, message):
        smtp.send(to, message)

class SendGridEmailSender(EmailSender):
    def send(self, to, message):
        sendgrid.send(to, message)

class LogEmailSender(EmailSender):
    def send(self, to, message):
        print(f"Mock email to {to}: {message}")

# High-level module
class UserRegistration:
    def __init__(self, email_sender: EmailSender):  # Depend on abstraction
        self.email_sender = email_sender

    def register_user(self, user):
        # ... register logic ...
        self.email_sender.send(user.email, "Welcome!")

# Easy to switch implementations
registration_prod = UserRegistration(SMTPEmailSender())
registration_test = UserRegistration(LogEmailSender())
registration_prod2 = UserRegistration(SendGridEmailSender())
```

Dependency Injection

DIP enables Dependency Injection:

```python
# Constructor injection
class OrderService:
    def __init__(self,
                 repository: OrderRepository,
```

```
                 payment: PaymentProcessor,
                 email: EmailSender):
        self.repository = repository
        self.payment = payment
        self.email = email

    def create_order(self, order):
        self.repository.save(order)
        self.payment.process(order)
        self.email.send(order.user.email, "Order confirmed")

# Inject dependencies
service = OrderService(
    repository=PostgreSQLRepository(),
    payment=StripePaymentProcessor(),
    email=SendGridEmailSender()
)

# For testing, inject mocks
test_service = OrderService(
    repository=MockRepository(),
    payment=MockPaymentProcessor(),
    email=MockEmailSender()
)
```

With Dependency Injection Container

```
# Using dependency_injector library
from dependency_injector import containers, providers

class Container(containers.DeclarativeContainer):
    # Database
    database = providers.Singleton(
        PostgreSQLDatabase,
        connection_string="postgresql://localhost/mydb"
    )

    # Repositories
    user_repository = providers.Factory(
        UserRepository,
        database=database
    )

    order_repository = providers.Factory(
        OrderRepository,
        database=database
    )

    # Services
    email_service = providers.Factory(EmailService)
```

```
    user_service = providers.Factory(
        UserService,
        repository=user_repository,
        email=email_service
    )

    order_service = providers.Factory(
        OrderService,
        repository=order_repository,
        email=email_service
    )

# Usage
container = Container()
user_service = container.user_service()
order_service = container.order_service()

# Both services share same database instance
# Easy to swap implementations
```

SOLID Principles Summary

Principle	What it means	Benefits
Single Responsibility	One class, one job	Easy to understand, test, maintain
Open/Closed	Extend, don't modify	Add features without breaking existing code
Liskov Substitution	Subtypes must be substitutable	Polymorphism works correctly
Interface Segregation	Small, focused interfaces	No forced unused methods
Dependency Inversion	Depend on abstractions	Easy to swap implementations, test

---

Complete Example: E-commerce System

Without SOLID (Bad)

```
class OrderProcessor:
    def __init__(self):
        pass

    def process_order(self, order):
        # Validate
        if not order.items:
            raise Exception("Empty order")

        # Calculate total
        total = sum(item.price * item.quantity for item in order.items)
```

```
        # Save to MySQL
        import mysql.connector
        db = mysql.connector.connect(host="localhost", database="orders")
        cursor = db.cursor()
        cursor.execute("INSERT INTO orders VALUES (?)", (total,))
        db.commit()

        # Process payment
        if order.payment_type == "credit_card":
            print("Processing credit card")
        elif order.payment_type == "paypal":
            print("Processing PayPal")

        # Send email
        import smtplib
        smtp = smtplib.SMTP('localhost')
        smtp.sendmail('noreply@example.com', order.user_email, 'Order confirmed')

        # Update inventory
        for item in order.items:
            cursor.execute("UPDATE products SET stock = stock - ? WHERE id = ?",
                           (item.quantity, item.product_id))
        db.commit()

        # Generate PDF invoice
        print("Generating PDF...")
```

Problems:

- OrderProcessor does everything (violates SRP)
- Can't extend payment types without modifying (violates OCP)
- Tightly coupled to MySQL (violates DIP)
- Can't test without database and email server

With SOLID (Good)

```
from abc import ABC, abstractmethod

# === ABSTRACTIONS (DIP) ===
class OrderRepository(ABC):
    @abstractmethod
    def save(self, order): pass

class PaymentProcessor(ABC):
    @abstractmethod
    def process(self, order): pass

class EmailSender(ABC):
    @abstractmethod
    def send(self, to, subject, body): pass
```

```python
class InventoryUpdater(ABC):
    @abstractmethod
    def update_stock(self, items): pass

class InvoiceGenerator(ABC):
    @abstractmethod
    def generate(self, order): pass

# === SINGLE RESPONSIBILITY ===
class Order:
    """Only holds order data"""
    def __init__(self, items, user_email, payment_type):
        self.items = items
        self.user_email = user_email
        self.payment_type = payment_type

class OrderValidator:
    """Only validates orders"""
    def validate(self, order):
        if not order.items:
            raise Exception("Empty order")
        return True

class OrderCalculator:
    """Only calculates totals"""
    def calculate_total(self, order):
        return sum(item.price * item.quantity for item in order.items)

# === OPEN/CLOSED (Payment types) ===
class CreditCardPayment(PaymentProcessor):
    def process(self, order):
        print("Processing credit card")
        return True

class PayPalPayment(PaymentProcessor):
    def process(self, order):
        print("Processing PayPal")
        return True

class CryptoPayment(PaymentProcessor):  # New type - no modification!
    def process(self, order):
        print("Processing crypto")
        return True

# === IMPLEMENTATIONS ===
class MySQLOrderRepository(OrderRepository):
    def save(self, order):
        # MySQL logic
        pass

class PostgreSQLOrderRepository(OrderRepository):
    def save(self, order):
        # PostgreSQL logic
        pass
```

```python
class SMTPEmailSender(EmailSender):
    def send(self, to, subject, body):
        # SMTP logic
        pass

class DatabaseInventoryUpdater(InventoryUpdater):
    def update_stock(self, items):
        # Update database
        pass

class PDFInvoiceGenerator(InvoiceGenerator):
    def generate(self, order):
        # Generate PDF
        pass

# === MAIN SERVICE (DIP - depends on abstractions) ===
class OrderService:
    def __init__(self,
                 validator: OrderValidator,
                 calculator: OrderCalculator,
                 repository: OrderRepository,
                 payment: PaymentProcessor,
                 email: EmailSender,
                 inventory: InventoryUpdater,
                 invoice: InvoiceGenerator):
        self.validator = validator
        self.calculator = calculator
        self.repository = repository
        self.payment = payment
        self.email = email
        self.inventory = inventory
        self.invoice = invoice

    def process_order(self, order):
        # Validate
        self.validator.validate(order)

        # Calculate
        total = self.calculator.calculate_total(order)

        # Save
        self.repository.save(order)

        # Payment
        self.payment.process(order)

        # Email
        self.email.send(order.user_email, "Order Confirmed", f"Total: ${total}")

        # Inventory
        self.inventory.update_stock(order.items)

        # Invoice
```

```
        self.invoice.generate(order)

# === USAGE ===
# Production
service = OrderService(
    validator=OrderValidator(),
    calculator=OrderCalculator(),
    repository=MySQLOrderRepository(),
    payment=CreditCardPayment(),
    email=SMTPEmailSender(),
    inventory=DatabaseInventoryUpdater(),
    invoice=PDFInvoiceGenerator()
)

# Testing
test_service = OrderService(
    validator=OrderValidator(),
    calculator=OrderCalculator(),
    repository=MockOrderRepository(),
    payment=MockPaymentProcessor(),
    email=MockEmailSender(),
    inventory=MockInventoryUpdater(),
    invoice=MockInvoiceGenerator()
)

# Different configuration
crypto_service = OrderService(
    validator=OrderValidator(),
    calculator=OrderCalculator(),
    repository=PostgreSQLOrderRepository(),  # Different DB
    payment=CryptoPayment(),  # Different payment
    email=SendGridEmailSender(),  # Different email
    inventory=DatabaseInventoryUpdater(),
    invoice=PDFInvoiceGenerator()
)
```

Benefits: ☑ Each class has one responsibility (SRP) ☑ Easy to add new payment types (OCP) ☑ Easy to swap implementations (DIP) ☑ Easy to test with mocks ☑ Clear, maintainable code

---

When to Apply SOLID

Apply When:

☑ Building production systems ☑ Code will be maintained long-term ☑ Multiple developers working together ☑ Requirements likely to change ☑ Need automated testing ☑ Reusability is important

Don't Over-Apply:

✗ Prototypes / POCs ✗ One-off scripts ✗ Very simple applications ✗ Stable, rarely-changing code ✗ When it adds complexity without benefit

Balance: Apply SOLID when benefits outweigh complexity

Common Mistakes

1. Over-Engineering

```
# TOO MUCH for simple use case
class StringValidator(ABC):
    @abstractmethod
    def validate(self, s: str): pass

class EmailValidator(StringValidator):
    def validate(self, s: str): pass

class PhoneValidator(StringValidator):
    def validate(self, s: str): pass

# Overkill for simple validation
# Just use functions:
def validate_email(email: str):
    return "@" in email
```

2. Premature Abstraction

```
# Don't create abstractions until you have 2+ implementations
# BAD: Only one implementation
class UserService(ABC):
    @abstractmethod
    def get_user(self): pass

class MySQLUserService(UserService):
    def get_user(self): pass  # Only implementation!

# GOOD: Start concrete, extract interface later
class UserService:
    def get_user(self): pass

# When you need second implementation, then extract interface
```

3. Interface Explosion

```
# TOO MANY interfaces
class Readable: pass
class Writable: pass
class Deletable: pass
class Updateable: pass
```

```
class Searchable: pass
class Sortable: pass
# ...50 more interfaces

# Better: Combine related operations
class Repository:
    def read(self): pass
    def write(self): pass
    def delete(self): pass
```

Interview Questions & Answers

Q: Explain SOLID principles

A: "SOLID is five design principles for maintainable OOP code:

- SRP: One class, one responsibility
- OCP: Extend, don't modify
- LSP: Subtypes must be substitutable
- ISP: Small, focused interfaces
- DIP: Depend on abstractions, not implementations

Example: Instead of one OrderProcessor doing everything (payment, email, database), split into OrderService, PaymentProcessor, EmailSender, OrderRepository. Each has one job, easy to test, and extend."

Q: When would you violate SOLID?

A: "For simple scripts, prototypes, or when following SOLID adds complexity without benefit. Example: A data migration script doesn't need dependency injection. But production systems benefit from SOLID for testability and maintainability."

Q: How does DIP help with testing?

A: "DIP means depending on interfaces, not implementations. In production, inject real database. In tests, inject mocks. Example:

```
# Production
service = UserService(database=PostgreSQL())

# Testing
service = UserService(database=MockDatabase())
```

Same code, different implementations. Easy to test without real database."

Q: Difference between LSP and ISP?

A: "LSP is about inheritance - subtypes must work wherever parent works. Penguin shouldn't inherit from FlyingBird because it can't fly.

ISP is about interfaces - don't force classes to implement methods they don't need. Robot shouldn't implement Eatable interface.

Both prevent violating contracts, but LSP is about inheritance, ISP is about interfaces."

Practical Tips

1. Start Simple, Refactor When Needed

```
# Start
class UserService:
    def get_user(self):
        db = connect_mysql()
        return db.query("SELECT * FROM users")

# Need testing? Refactor to DIP
class UserService:
    def __init__(self, db):
        self.db = db

    def get_user(self):
        return self.db.query("SELECT * FROM users")

# Need multiple databases? Extract interface
class Database(ABC):
    @abstractmethod
    def query(self, sql): pass
```

2. Use Type Hints

```
# Makes dependencies explicit
class OrderService:
    def __init__(self,
                 repository: OrderRepository,
                 payment: PaymentProcessor):
        self.repository = repository
        self.payment = payment
```

3. Dependency Injection Frameworks

```
# For large applications
from dependency_injector import containers, providers

class Container(containers.DeclarativeContainer):
    database = providers.Singleton(PostgreSQL)
    repository = providers.Factory(UserRepository, db=database)
```

```
    service = providers.Factory(UserService, repo=repository)

container = Container()
service = container.service()
```

4. Test-Driven Development

```
# Write tests first, SOLID naturally emerges
def test_user_service():
    mock_db = MockDatabase()
    service = UserService(mock_db)  # DIP naturally applied

    user = service.get_user(123)
    assert user.name == "Alice"
```

Tools and Resources

Static Analysis:

- Pylint (Python)
- SonarQube (multi-language)
- ESLint (JavaScript)

Design Patterns:

- Factory Pattern (OCP)
- Strategy Pattern (OCP)
- Adapter Pattern (DIP)
- Decorator Pattern (OCP)

Books:

- Clean Architecture (Robert C. Martin)
- Clean Code (Robert C. Martin)
- Design Patterns (Gang of Four)

Learning:

- Practice with refactoring exercises
- Code reviews focusing on SOLID
- Start with SRP, gradually add others

Summary

SOLID makes code:

- ☑ Easier to understand (SRP)

- ☑ Easier to extend (OCP)
- ☑ Safer to change (LSP)
- ☑ More focused (ISP)
- ☑ Easier to test (DIP)

Remember:

- Apply pragmatically, not dogmatically
- Start simple, refactor when needed
- Balance SOLID with simplicity
- Use when benefits outweigh complexity

Key Takeaway: SOLID principles guide you toward maintainable, flexible, testable code. Master them through practice, not memorization.